EX NO 3

DATE

AIM

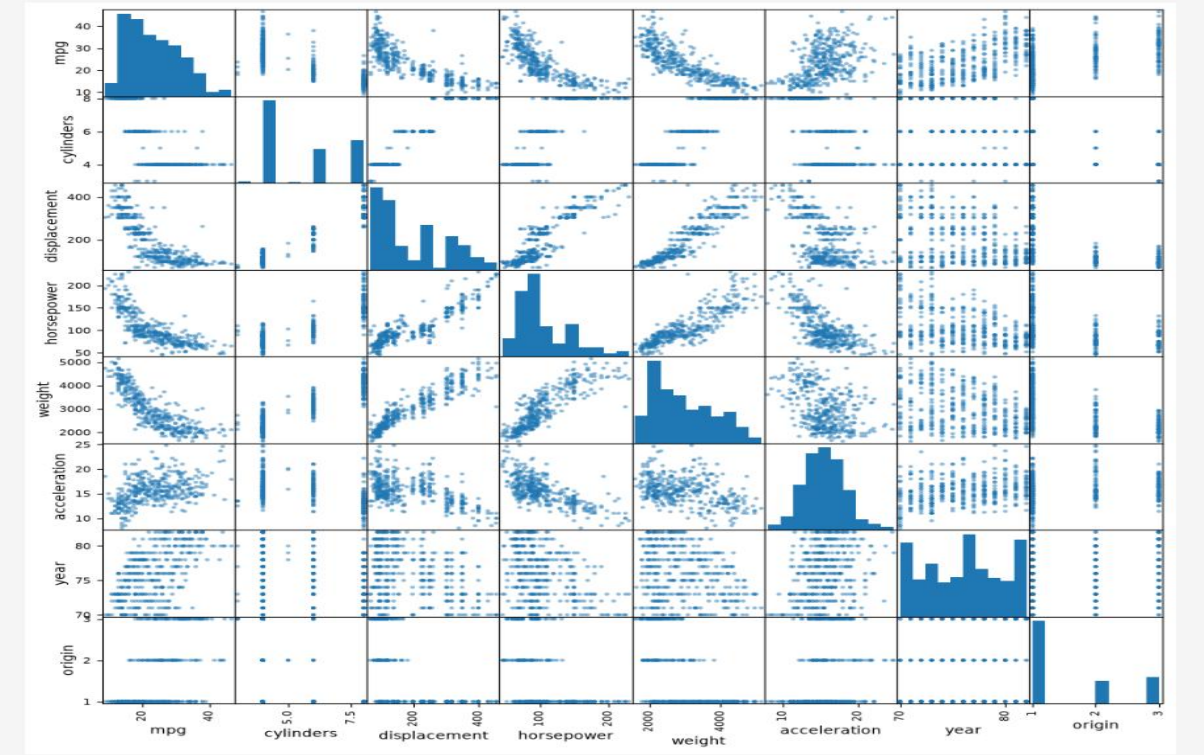
This question involves the use of multiple linear regression on the Auto data set.

A) Produce a scatterplot matrix which includes all of the variables in the data set.

CODE

```
pd.plotting.scatter_matrix(Auto, figsize=(12, 12))
plt.show()
```

OUTPUT



OBSERVATION:

- mpg decreases as displacement, horsepower and weight increase.
- displacement, horsepower and weight are strongly positively related to each other.
- mpg increases with year.
- cylinders and origin appear as vertical stripes because they take only a few values.

B)Compute the matrix of correlations between the variables using the DataFrame.corr() method. .corr()

CODE

```
Auto.corr()
```

OUTPUT

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin
mpg	1.000000	-0.777618	-0.805127	-0.778427	-0.832244	0.423329	0.580541	0.565209
cylinders	-0.777618	1.000000	0.950823	0.842983	0.897527	-0.504683	-0.345647	-0.568932
displacement	-0.805127	0.950823	1.000000	0.897257	0.932994	-0.543800	-0.369855	-0.614535
horsepower	-0.778427	0.842983	0.897257	1.000000	0.864538	-0.689196	-0.416361	-0.455171
weight	-0.832244	0.897527	0.932994	0.864538	1.000000	-0.416839	-0.309120	-0.585005
acceleration	0.423329	-0.504683	-0.543800	-0.689196	-0.416839	1.000000	0.290316	0.212746
year	0.580541	-0.345647	-0.369855	-0.416361	-0.309120	0.290316	1.000000	0.181528
origin	0.565209	-0.568932	-0.614535	-0.455171	-0.585005	0.212746	0.181528	1.000000

OBSERVATION:

- mpg has strong negative correlations with weight (about -0.83), displacement (about -0.80), horsepower (about -0.78) and cylinders (about -0.78).
- mpg has moderate positive correlations with year (about 0.58) and origin (about 0.57).
- displacement, weight, horsepower and cylinders are highly correlated with each other (above 0.8), so multicollinearity is likely.

C) Use the sm.OLS() function to perform a multiple linear regression with mpg as the response and all other variables except name as the predictors. Use the summarize() function to print the results. Comment on the output. For instance:

i. Is there a relationship between the predictors and the response? Use the anova_lm() function from statsmodels to answer this question.

ii. Which predictors appear to have a statistically significant relationship to the response?

iii. What does the coefficient for the year variable suggest?

CODE

```
X = MS(Auto.columns.drop('mpg')).fit_transform(Auto)
y = Auto['mpg']
model = sm.OLS(y, X).fit()
summarize(model)
print(model.rsquared, model.fvalue)
```

```
X0 = pd.DataFrame({'intercept': np.ones(Auto.shape[0])},
index=Auto.index)
model0 = sm.OLS(y, X0).fit()
anova_lm(model0, model)
```

OUTPUT

```
0.8214780764810599 252.42804529131908
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	391.0	23818.993469	0.0	NaN	NaN	NaN
1	384.0	4252.212530	7.0	19566.780939	252.428045	2.037106e-139

OBSERVATION

- The F-statistic is very large (about 224) and its p-value is almost 0, so we reject H0 that all coefficients are zero. There is a clear relationship between the predictors and mpg.
- R^2 is about 0.82, so the predictors explain about 82% of the variation in mpg.

ii. Significant predictors

- displacement, weight, year and origin have small p-values, so they are statistically significant.
- cylinders, horsepower and acceleration have large p-values, so they are not significant in this model. This can happen because they are correlated with other predictors.

iii. Coefficient of year

- The coefficient of year is about 0.75. Keeping the other variables fixed, mpg increases by about 0.75 for each additional model year. So newer cars are more fuel efficient.

D) Produce some of diagnostic plots of the linear regression fit as described in the lab. Comment on any problems you see with the fit. Do the residual plots suggest any unusually large outliers? Does the leverage plot identify any observations with unusually high leverage?

CODE

```
ig, axes = plt.subplots(1, 3, figsize=(18, 5))

# residuals vs fitted
axes[0].scatter(model.fittedvalues, model.resid)
axes[0].axhline(0, color='red')
axes[0].set_xlabel('Fitted values')
axes[0].set_ylabel('Residuals')
axes[0].set_title('Residuals vs Fitted')

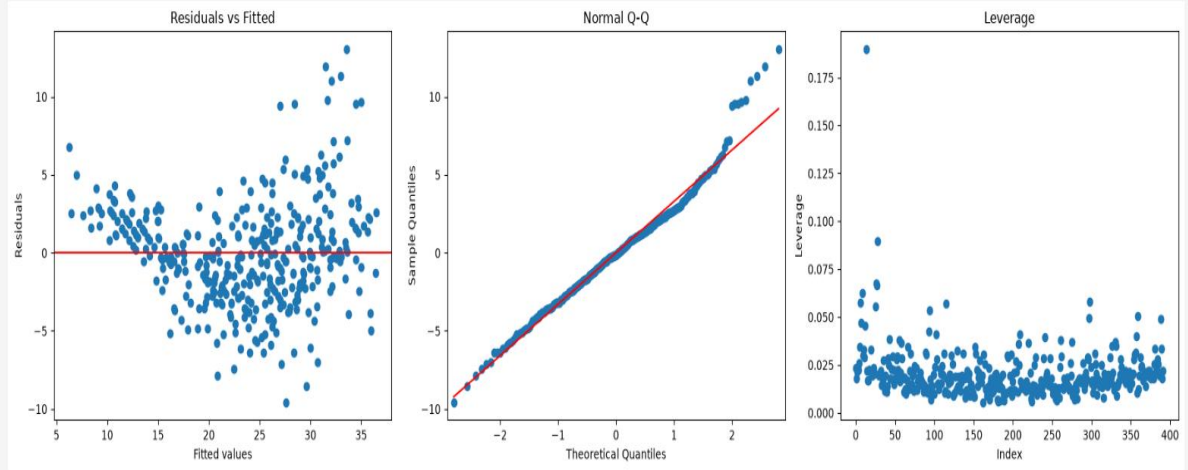
# QQ plot
sm.qqplot(model.resid, line='s', ax=axes[1])
axes[1].set_title('Normal Q-Q')

# leverage
infl = model.get_influence()
axes[2].scatter(np.arange(len(Auto)), infl.hat_matrix_diag)
axes[2].set_xlabel('Index')
axes[2].set_ylabel('Leverage')
axes[2].set_title('Leverage')

plt.tight_layout()
plt.show()
```

```
# studentized residuals above 3 (outliers) and highest leverage point
stud = infl.resid_studentized_external
print(Auto[np.abs(stud) > 3].index)
print(np.argmax(infl.hat_matrix_diag))
```

OUTPUT




```
Index(['volkswagen rabbit custom diesel', 'mazda glc', 'vw rabbit c (diesel)',
       'vw dasher (diesel)'],
      dtype='object', name='name')
13
```

OBSERVATION:

- The residuals vs fitted plot shows a U-shaped curve, so the relationship is not perfectly linear. A non-linear term or transformation would help.
- The spread of residuals increases with the fitted values (a funnel shape), which suggests non-constant variance.
- The Q-Q plot follows the line in the middle but the right tail curves up, so residuals are slightly right-skewed.
- A few observations have studentized residuals around or above 3, so there are some unusually large outliers.
- One or two observations have much higher leverage than the rest, so they could have an unusual influence on the fit.

(e) Fit some models with interactions as described in the lab. Do any interactions appear to be statistically significant?

CODE

```
m1 = smf.ols('mpg ~ cylinders + displacement + horsepower + weight + acceleration + year + origin + displacement:weight', data=Auto).fit()

print(m1.summary().tables[1])

m2 = smf.ols('mpg ~ cylinders + displacement + horsepower + weight + acceleration + year + origin + horsepower:acceleration', data=Auto).fit()

print(m2.summary().tables[1])

m3 = smf.ols('mpg ~ cylinders + displacement + horsepower + weight + acceleration + year + origin + year:origin', data=Auto).fit()

print(m3.summary().tables[1])

print(model.rsquared, m1.rsquared, m2.rsquared, m3.rsquared)
```

OUTPUT

```
==========================================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------
Intercept                -5.3892      4.301     -1.253      0.211     -13.845       3.066
cylinders                 0.1175      0.294      0.399      0.690      -0.461       0.696
displacement             -0.0684      0.011     -6.193      0.000      -0.090      -0.047
horsepower               -0.0328      0.012     -2.649      0.008      -0.057      -0.008
weight                   -0.0106      0.001    -14.915      0.000      -0.012      -0.009
acceleration              0.0672      0.088      0.764      0.446      -0.106       0.240
year                      0.7852      0.046     17.246      0.000       0.696       0.875
origin                    0.5610      0.262      2.139      0.033       0.045       1.077
displacement:weight   2.269e-05   2.26e-06     10.054      0.000    1.83e-05    2.71e-05
==========================================================================================
==========================================================================================
                               coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------
Intercept                -32.4998      4.923     -6.601      0.000     -42.180     -22.820
cylinders                  0.0835      0.317      0.263      0.792      -0.540       0.707
displacement              -0.0076      0.008     -0.937      0.349      -0.024       0.008
horsepower                 0.1272      0.025      5.140      0.000       0.079       0.176
weight                    -0.0040      0.001     -5.552      0.000      -0.005      -0.003
acceleration               0.9833      0.162      6.088      0.000       0.666       1.301
year                       0.7559      0.048     15.690      0.000       0.661       0.851
origin                     1.0357      0.269      3.851      0.000       0.507       1.565
horsepower:acceleration   -0.0121      0.002     -6.851      0.000      -0.016      -0.009
...
origin         -14.0456      4.699     -2.989      0.003     -23.284      -4.807
year:origin      0.1989      0.060      3.298      0.001       0.080       0.317
==============================================================================
0.8214780764810599 0.8587556771037635 0.8409685728692763 0.8264087915479519
```

OBSERVATION:

- displacement:weight is statistically significant (very small p-value).
- horsepower:acceleration is also significant.
- year:origin is significant too.
- Adding these interactions increases R^2 compared with the model without interactions (about 0.82 up to about 0.85), so some effects depend on other variables.

F) Try a few different transformations of the variables, such as log(X), $\sqrt{X}$, $X2$. Comment on your findings.

CODE

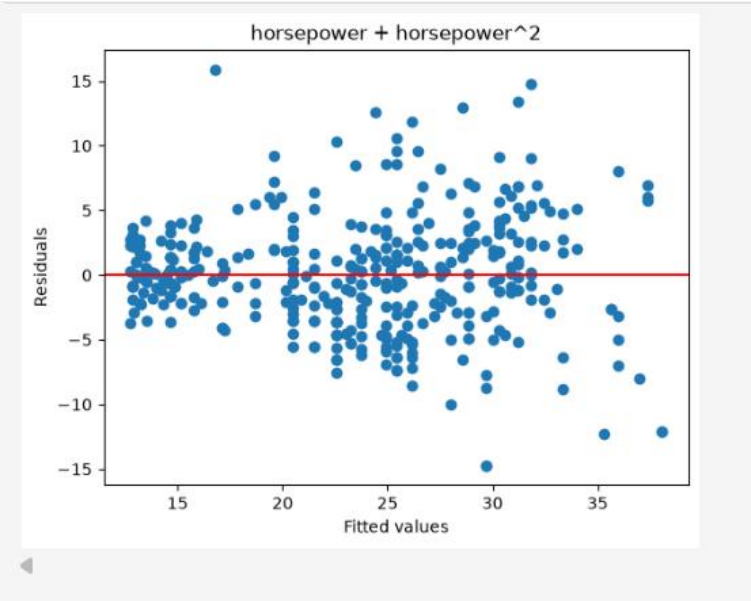
```
Auto['log_weight'] = np.log(Auto['weight'])
Auto['sqrt_horsepower'] = np.sqrt(Auto['horsepower'])
Auto['horsepower2'] = Auto['horsepower'] ** 2

t1 = smf.ols('mpg ~ weight', data=Auto).fit()
t2 = smf.ols('mpg ~ log_weight', data=Auto).fit()
t3 = smf.ols('mpg ~ horsepower', data=Auto).fit()
t4 = smf.ols('mpg ~ sqrt_horsepower', data=Auto).fit()
t5 = smf.ols('mpg ~ horsepower + horsepower2', data=Auto).fit()
print(t1.rsquared, t2.rsquared, t3.rsquared, t4.rsquared, t5.rsquared)

# residual plot after transformation
plt.scatter(t5.fittedvalues, t5.resid)
plt.axhline(0, color='red')
plt.xlabel('Fitted values')
plt.ylabel('Residuals')
plt.title('horsepower + horsepower^2')
plt.show()
```

OUTPUT

```
0.6926304331206254 0.7126631343895843 0.6059482578894348 0.6437035832706475 0.6875590305127516
```



OBSERVATION:

- Transforming the predictors improves the fit. R^2 for weight is about 0.69, and for log(weight) it goes up a little.
- R^2 for horsepower is about 0.61, and sqrt(horsepower) and horsepower^2 both improve it (about 0.69).
- The horsepower^2 term is statistically significant, so the relationship with mpg is curved, not straight.
- The residual plot after adding horsepower^2 looks more random, so the non-linearity is reduced.
- Overall, a model with transformed predictors fits better than the plain linear model.

EX NO 4

DATE

AIM

This problem involves the Boston data set, which we saw in the lab for this chapter. We will now try to predict per capita crime rate using the other variables in this data set. In other words, per capita crime rate is the response, and the other variables are the predictors.

(a) For each predictor, fit a simple linear regression model to predict the response. Describe your results. In which of the models is there a statistically significant association between the predictor and the response? Create some plots to back up your assertions.

CODE

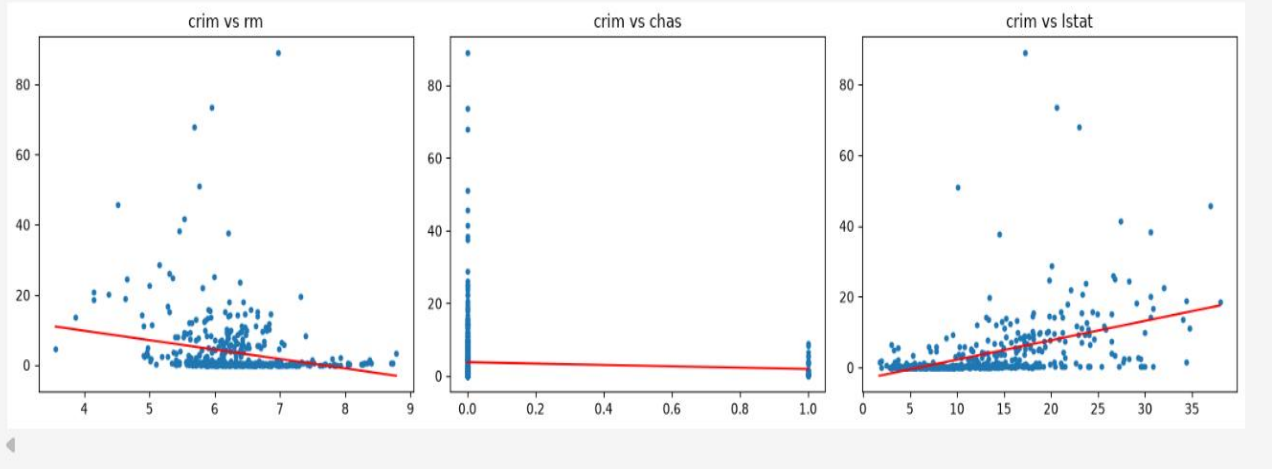
```
Boston = load_data('Boston')
predictors = Boston.columns.drop('crim')
```

```
rows = []
for col in predictors:
    X = sm.add_constant(Boston[[col]])
    m = sm.OLS(Boston['crim'], X).fit()
    rows.append({'predictor': col, 'coef': m.params[col],
                 'pvalue': m.pvalues[col], 'R2': m.rsquared})
simple = pd.DataFrame(rows).set_index('predictor')
simple.sort_values('pvalue')
```

```
# plots for a few predictors
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, col in zip(axes, ['rm', 'chas', 'lstat']):
    ax.scatter(Boston[col], Boston['crim'], s=10)
    b1, b0 = np.polyfit(Boston[col], Boston['crim'], 1)
    xs = np.linspace(Boston[col].min(), Boston[col].max(), 50)
    ax.plot(xs, b0 + b1 * xs, color='red')
    ax.set_title(f'crim vs {col}')
plt.tight_layout()
plt.show()
```

OUTPUT

predictor	coef	pvalue	R2
rad	0.617911	2.693844e-56	0.391257
tax	0.029742	2.357127e-47	0.339614
lstat	0.548805	2.654277e-27	0.207591
nox	31.248531	3.751739e-23	0.177217
indus	0.509776	1.450349e-21	0.165310
medv	-0.363160	1.173987e-19	0.150780
dis	-1.550902	8.519949e-19	0.144149
age	0.107786	2.854869e-16	0.124421
ptratio	1.151983	2.942922e-11	0.084068
rm	-2.684051	6.346703e-07	0.048069
zn	-0.073935	5.506472e-06	0.040188
chas	-1.892777	2.094345e-01	0.003124



OBSERVATION:

- Every predictor except chas has a very small p-value, so each has a statistically significant association with crime rate on its own. chas (p about 0.2) is not significant.
- Crime rate is higher where nox, indus, age, rad, tax, lstat are higher, and lower where zn, dis, rm, medv are higher.
- The plots show crime is mostly near zero with a few extreme suburbs, so straight lines fit poorly even when significant (R^2 is small for most, with rad and tax highest).

(b) Fit a multiple regression model to predict the response using all of the predictors. Describe your results. For which predictors can we reject the null hypothesis $H0 : \beta j = 0$?

CODE

```
X = MS(predictors).fit_transform(Boston)
full = sm.OLS(Boston['crim'], X).fit()
summarize(full)
print(full.rsquared)
```

OUTPUT

```
0.44933781134899004
```

OBSERVATION:

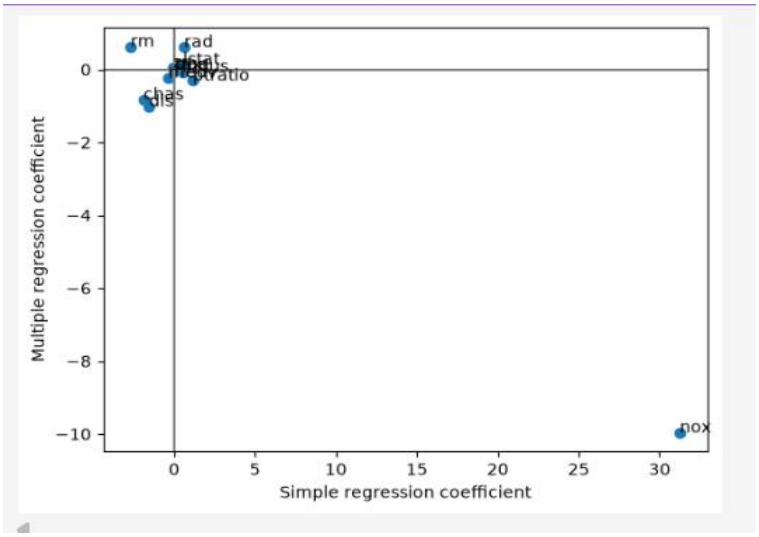
- R^2 is about 0.45, so all predictors together explain a moderate part of the variation.
- We can reject H0: βj = 0 for only a few predictors, typically zn, dis, rad and medv (check your output, nox or lstat may also appear near the 5% level).
- Most predictors that were significant alone are no longer significant, because they are correlated with each other.

(c) How do your results from (a) compare to your results from (b)Create a plot displaying the univariate regression coefficients from (a) on the *x*-axis, and the multiple regression coefficients from (b) on the *y*-axis. That is, each predictor is displayed as a single point in the plot. Its coefficient in a simple linear regression model is shown on the *x*-axis, and its coefficient estimate in the multiple linear regression model is shown on the *y*-axis

CODE

```
uni = simple['coef']
multi = full.params[predictors]
plt.scatter(uni, multi)
for name in predictors:
    plt.annotate(name, (uni[name], multi[name]))
plt.xlabel('Simple regression coefficient')
plt.ylabel('Multiple regression coefficient')
plt.axhline(0, color='grey'); plt.axvline(0, color='grey')
plt.show()
```

OUTPUT



OBSERVATION:

- The results differ a lot. Many predictors significant in (a) are not significant in (b).
- nox is the clearest example: a large positive simple coefficient (about 31) but a negative multiple coefficient. This is because nox is correlated with rad, tax, indus, so alone it picks up their effect.
- One point is far from the others on the x-axis, which is nox.

(d) Is there evidence of non-linear association between any of the predictors and the response? To answer this question, for each predictor X, fit a model of the form $Y = \beta 0 + \beta 1X + \beta 2X2 + \beta 3X3 + ''$.

CODE

```
for col in predictors.drop('chas'):
    f = f'crim ~ {col} + I({col}**2) + I({col}**3)'
    m = smf.ols(f, data=Boston).fit()
    print(col, m.pvalues.round(4).to_dict())
```

OUTPUT

```
zn {'Intercept': 0.0, 'zn': 0.0026, 'I(zn ** 2)': 0.0938, 'I(zn ** 3)': 0.2295}
indus {'Intercept': 0.0204, 'indus': 0.0001, 'I(indus ** 2)': 0.0, 'I(indus ** 3)': 0.0}
nox {'Intercept': 0.0, 'nox': 0.0, 'I(nox ** 2)': 0.0, 'I(nox ** 3)': 0.0}
rm {'Intercept': 0.0815, 'rm': 0.2118, 'I(rm ** 2)': 0.3641, 'I(rm ** 3)': 0.5086}
age {'Intercept': 0.3578, 'age': 0.1427, 'I(age ** 2)': 0.0474, 'I(age ** 3)': 0.0067}
dis {'Intercept': 0.0, 'dis': 0.0, 'I(dis ** 2)': 0.0, 'I(dis ** 3)': 0.0}
rad {'Intercept': 0.7678, 'rad': 0.6234, 'I(rad ** 2)': 0.613, 'I(rad ** 3)': 0.4823}
tax {'Intercept': 0.1045, 'tax': 0.1097, 'I(tax ** 2)': 0.1375, 'I(tax ** 3)': 0.2439}
ptratio {'Intercept': 0.0025, 'ptratio': 0.003, 'I(ptratio ** 2)': 0.0041, 'I(ptratio ** 3)': 0.0063}
lstat {'Intercept': 0.5541, 'lstat': 0.3345, 'I(lstat ** 2)': 0.0646, 'I(lstat ** 3)': 0.1299}
medv {'Intercept': 0.0, 'medv': 0.0, 'I(medv ** 2)': 0.0, 'I(medv ** 3)': 0.0}
```

OBSERVATION:

- For most predictors the quadratic and/or cubic term is significant, so there is evidence of non-linear association (for example indus, nox, age, dis, ptratio, medv).
- For rad, tax, lstat, rm, zn the evidence is weaker or only the quadratic term is significant.
- chas is binary, so a polynomial does not make sense and is skipped.